```
''' Q1) What is the significance of classes in Python programming.and how do they contribute to object oriented programming '''

''' Classes are fundamental to Python's object-oriented programming (OOP) paradigm, acting as blueprints for creating objects tha
reusable, and scalable code '''
```

```
''' Q2) Create a custom Python class for managing a bank account with basic functionalities like deposit and
withdrawal?'''
class Bank:
    def __init__(self,amount):
        self.amount=amount
    def withdraw(self,amt):
        data=self.amount-amt
        if(data<0):
            print("Amount cannnt be withdrawn")
        else:
            self.amount=data
            print("Amount withdawn successfully ",self.amount)

    def deposit(self,amt):
        if(amt<0):
           print("please enter a valid deposit")
        else:
            self.amount+=amt
            print("Amount deposited successfully ",self.amount)
b1 = Bank(2000)
b1.deposit(1000)
b1.withdraw(2000)
```

```
Amount deposited successfully  3000
Amount withdawn successfully  1000
```

```
'''Q3) Create a Book class that contains multiple Chapters, where each Chapter has a title and page count.
Write code to initialize a Book object with three chapters and display the total page count of the book.'''

class Chapter:
    def __init__(self,title,pc):
        self.title=title
        self.pc=pc
class Book:
    def __init__(self,chapters):
        self.chapters=chapters

    def getTotalPage(self):
        totalPc=0
        for i in self.chapters:
            totalPc+=i.pc
        return totalPc
ch1=Chapter("The wind and sun",123)
ch2=Chapter("You can win",200)
ch3 = Chapter("With Love",300)
b = Book([ch1,ch2,ch3])
print("The total page is ",b.getTotalPage())
```

```
The total page is  623
```

```
''' Q4) How does python enforce access control to class attribute ,and what is the difference between public protected and priva

''' Python does not enforce true access control in the way languages like Java or C++ do; instead, it uses a convention-based sy
to signify intended visibility
The convention defines three main levels of visibility:

1.Public attributes have no leading underscores and are intended for use by any part of the program
2.Protected attributes are prefixed with a single underscore (e.g., _attribute_name) . This convention serves as a hint to other
that the attribute is intended for internal use within the class or its subclasses, but it does not prevent external access.
3.Private attributes are prefixed with a double underscore (e.g., __attribute_name).
This triggers a mechanism called "name mangling," where the attribute name is modified internally
(e.g., to _ClassName__attribute_name). This makes external access more difficult
and less direct, but not impossible, thus "pseudo-private"'''
```

```
''' Q5) Write a Python program using a Time class to input a given time in 24-hour format and convert it
to a 12-hour format with AM/PM. The program should also validate time strings to ensure they are
in the correct HH:MM:SS format. Implement a method to check if the time is valid and return an
appropriate message. '''
```

```
class Time:
    def __init__(self, time_str):
        self.time_str = time_str

    def is_valid(self):
        parts = self.time_str.split(":")


        if len(parts) != 3:
            return False, "Invalid format! Use HH:MM:SS"

        try:
            hh, mm, ss = map(int, parts)
        except ValueError:
            return False, "Time must contain only numbers!"


        if not (0 <= hh <= 23):
            return False, "Hour must be between 00 and 23"
        if not (0 <= mm <= 59):
            return False, "Minute must be between 00 and 59"
        if not (0 <= ss <= 59):
            return False, "Second must be between 00 and 59"

        return True, "Valid time"

    def convert12Hr(self):
        valid, message = self.is_valid()
        if not valid:
            return message

        hh, mm, ss = map(int, self.time_str.split(":"))

        period = "AM"
        if hh >= 12:
            period = "PM"


        if hh == 0:
            hh = 12
        elif hh > 12:
            hh -= 12

        return f"{hh:02d}:{mm:02d}:{ss:02d} {period}"


time_input = input("Enter time (HH:MM:SS): ")
t = Time(time_input)

valid, msg = t.is_valid()
if valid:
    print("12-hour format:", t.convert12Hr())
else:
    print("Error:", msg)


#Input:
#Enter time (HH:MM:SS):  23:02:30
```

```
Enter time (HH:MM:SS):  23:02:30
12-hour format: 11:02:30 PM
```

```
''' Q6) Write a Python program that uses private attributes for creating a BankAccount class. Implement
methodstodeposit, withdraw, and display the balance, ensuring direct access to the balance attribute is
restricted. Explain why using private attributes can help improve data security and prevent accidental
modifications '''
class BankAccount:
    def __init__(self,accId,balance):
        self.__accId=accId
        self.__balance=balance
    def withdraw(self,amt):
        data=self.__balance-amt
        if(data<0):
            print("Amount cannnt be withdrawn")
        else:
            self.__balance=data
```

```
            print("Amount withdawn successfully total amount after withdraw ",self.__balance)

    def deposit(self,amt):
        if(amt<0):
           print("please enter a valid deposit")
        else:
            self.__balance+=amt
            print("Amount deposited successfully total amount after deposit ",self.__balance)

b1 = BankAccount(123,2340)
b1.withdraw(120)
b1.deposit(1200)
```

```
Amount withdawn successfully total amount after withdraw  2220
Amount deposited successfully total amount after deposit  3420
```

```
'''Q7) Write a Python program to simulate a card game using object-oriented principles. The program should
include a Card class to represent individual playing cards, a Deck class to represent a deck of cards,
and a Player class to represent players receiving cards. Implement a shuffle method in the Deck class
to shuffle the cards and a deal method to distribute cards to players. Display each player’s hand after
dealing.'''

import random
class Card:
    def __init__(self, suit, rank):
        self.suit = suit
        self.rank = rank

    def __str__(self):
        return f"{self.rank} of {self.suit}"
class Deck:
    def __init__(self):
        suits = ['Hearts', 'Diamonds', 'Clubs', 'Spades']
        ranks = ['2', '3', '4', '5', '6', '7', '8', '9', '10', 'Jack', 'Queen', 'King', 'Ace']

        self.cards = [Card(s, r) for s in suits for r in ranks]

    def shuffle(self):
        random.shuffle(self.cards)

    def deal(self, num_cards):

        hand = []
        for _ in range(num_cards):
            if self.cards:
                hand.append(self.cards.pop())
        return hand

class Player:
    def __init__(self, name):
        self.name = name
        self.hand = []

    def show_hand(self):
        print(f"\n{self.name}'s Hand:")
        for card in self.hand:
            print(f"  - {card}")

my_deck = Deck()
my_deck.shuffle()

player1 = Player("Amrita")
player2 = Player("Bob")


player1.hand = my_deck.deal(5)
player2.hand = my_deck.deal(5)


player1.show_hand()
player2.show_hand()
```

```
Amrita's Hand:
  - 2 of Hearts
  - Ace of Diamonds
  - Queen of Spades
```

```
  - 8 of Hearts
  - 4 of Spades

Bob's Hand:
  - 3 of Hearts
  - 7 of Spades
  - 10 of Hearts
  - 9 of Diamonds
  - Queen of Hearts
```

```
'''8) Write a Python program that defines a base class Vehicle with attributes make and model, and a
method display info(). Create a subclass Car that inherits from Vehicle and adds an additional at-
tribute num doors. Instantiate both Vehicle and Car objects, call their display info() methods, and
explain how the subclass inherits and extends the functionality of the base class.'''
class Vehicle:
    def __init__(self,make,model):
        self.make=make
        self.model=model

    def display_info(self):
        print("Make:",self.make)
        print("Model:",self.model)

class Car(Vehicle):
    def __init__(self, make, model,num_doors):
        super().__init__(make, model)
        self.num_doors=num_doors

    def display_info(self):
        super().display_info()
        print("Number of Doors:",self.num_doors)

v1 =Vehicle("Toyota","ABC")
c1=Car("Honda","DEF",4)
print("Vehicle info:")
v1.display_info()
print("Car info:")
c1.display_info()
```

```
Vehicle info:
Make: Toyota
Model: ABC
Car info:
Make: Honda
Model: DEF
Number of Doors: 4
```

```
'''9) Write a Python program demonstrating polymorphism by creating a base class Shape with a method
area(), and two subclasses Circle and Rectangle that override the area() method. Instantiate objects
of both subclasses and call the area() method. Explain how polymorphism simplifies working with
different shapes in an inheritance hierarchy.
'''
import math
class Shape:
    def area(self):
         print("Area")
class Circle(Shape):
     def __init__(self,rad):
          self.rad = rad

     def area(self):
          return math.pi*self.rad*self.rad
class Rectangle(Shape):
     def __init__(self,length,breadth):
          self.length=length
          self.breadth=breadth

     def area(self):
        return self.length*self.breadth

c1 = Circle(15)
print("Area of circle:",c1.area())

r1 = Rectangle(7,8)
print("Area of circle:",r1.area())
```

```
'''Polymorphism (meaning "many forms") in the context of this program is the ability of objects of different classes to respond
to the same method call (area()) in different, class-specific ways.
In this inheritance hierarchy:
Common Interface: The base Shape class defines a common interface for its subclasses through the area() method.
Overriding Behavior: Both Circle and Rectangle override this method, providing their own specific implementation for calculatin
```

```
Area of circle: 706.8583470577034
Area of circle: 56
```

```
'''10) Implement the CommissionEmployee class with init , earnings, and repr methods. Include
properties for personal details and sales data. Create a test script to instantiate the object, display
earnings, modify sales data, and handle data validation errors for negative values.'''

class CommissionEmployee:
    def __init__(self, first_name, last_name, ssn, gross_sales, commission_rate):
        self.first_name = first_name
        self.last_name = last_name
        self.ssn = ssn
        self.__gross_sales = gross_sales
        self.__commission_rate = commission_rate

    @property
    def gross_sales(self):
        return self.__gross_sales

    @gross_sales.setter
    def gross_sales(self, value):
        if value < 0:
            raise ValueError("Gross sales cannot be negative.")
        self.__gross_sales = value

    @property
    def commission_rate(self):
        return self.__commission_rate

    @commission_rate.setter
    def commission_rate(self, value):
        if value < 0 or value > 1:
            raise ValueError("Commission rate must be between 0 and 1.")
        self.__commission_rate = value

    def earnings(self):
        return self.__gross_sales * self.__commission_rate

    def __repr__(self):
        return (f"CommissionEmployee({self.first_name} {self.last_name}, "
                f"SSN: {self.ssn}, Gross Sales: {self.gross_sales}, "
                f"Commission Rate: {self.__commission_rate})")


def main():
    try:
        emp = CommissionEmployee("Akash", "Gupta", "123-45-6789", 50000, 0.1)
        print(emp)
        print("Earnings:", emp.earnings())
        print("\nUpdating gross sales to 70000...")
        emp.gross_sales = 70000
        print("Updated Earnings:", emp.earnings())
        print("\nTrying to set negative sales...")
        emp.gross_sales = -1000

    except ValueError as e:
        print("Error:", e)


if __name__ == "__main__":
    main()
```

```
CommissionEmployee(Akash Gupta, SSN: 123-45-6789, Gross Sales: 50000, Commission Rate: 0.1)
Earnings: 5000.0

Updating gross sales to 70000...
Updated Earnings: 7000.0

Trying to set negative sales...
Error: Gross sales cannot be negative.
```

```
'''Q11) What is duck typing in Python? Write a Python program demonstrating duck typing by creating a
function describe() that accepts any object with a speak() method. Implement two classes, Dog and
Robot, each with a speak() method. Pass instances of both classes to the describe() function and
explain how duck typing allows the function to work without checking the object’s type.'''


'''Duck typing in Python is a programming concept where the suitability of an object is determined by its methods and propertie
rather than its explicit type or class inheritance'''

def describe(obj):
    obj.speak()

class Dog:
    def speak(self):
        print("Dog barks")

class Robot:
    def speak(self):
        print("Robot says Hi!")

d1 = Dog()
r1 = Robot()
describe(d1)
describe(r1)
```

```
Dog barks
Robot says Hi!
```

```
'''Q12. WAP to overload the + operator to perform addition of two complex numbers using a custom Complex
class?'''

class Complex:
    def __init__(self, real, imag):
        self.real = real
        self.imag = imag
    def __add__(self, other):
        return Complex(self.real + other.real,
                       self.imag + other.imag)

    def __repr__(self):
        return f"{self.real} + {self.imag}i"

c1 = Complex(2,3)
c2 = Complex(4,5)
result = c1+c2
print("Result:", result)
```

```
Result: 6 + 8i
```

```
'''13. WAP to create a custom exception class in Python that displays the balance and withdrawal amount
when an error occurs due to insufficient funds?'''
class InsufficientFundsError(Exception):
    def __init__(self, balance, withdraw_amount):
        self.balance = balance
        self.withdraw_amount = withdraw_amount
        super().__init__(f"Insufficient funds! Balance: {balance}, Withdrawal: {withdraw_amount}")
class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientFundsError(self.balance, amount)
        self.balance -= amount
        print("Withdrawal successful! Remaining balance:", self.balance)
try:
    acc = BankAccount(5000)
    acc.withdraw(7000)
except InsufficientFundsError as e:
    print("Error:", e)
```

```
Error: Insufficient funds! Balance: 5000, Withdrawal: 7000
```

```
'''14. Write a Python program using the Card data class to simulate dealing 5 cards to a player from a
shuffled deck of standard playing cards. The program should print the player’s hand and the number
```

```
of remaining cards in the deck after the deal.'''

import random
from dataclasses import dataclass

@dataclass
class Card:
    suit: str
    rank: str

suits = ["Hearts", "Diamonds", "Clubs", "Spades"]
ranks = ["2", "3", "4", "5", "6", "7", "8", "9", "10",
         "J", "Q", "K", "A"]

deck = [Card(suit, rank) for suit in suits for rank in ranks]
random.shuffle(deck)

hand = deck[:5]
remaining_deck = deck[5:]

print("Player's Hand:")
for card in hand:
    print(f"{card.rank} of {card.suit}")

print("\nRemaining cards:", len(remaining_deck))
```

```
Player's Hand:
10 of Clubs
5 of Hearts
3 of Diamonds
2 of Spades
7 of Clubs

Remaining cards: 47
```

```
'''15. How do Python data classes provide advantages over named tuples in terms of flexibility and func-
tionality? Give an example using python code.'''

#Data Class
from dataclasses import dataclass
@dataclass
class Student:
    name: str
    age: int = 18  # default value

    def greet(self):
        return f"Hello, I am {self.name}, Age: {self.age}"

s = Student("Ankit")
print(s)
s.age = 20  # mutable
print(s.greet())
```

```
Student(name='Ankit', age=18)
Hello, I am Ankit, Age: 20
```

```
# 16. Write a Python program that demonstrates unit testing directly within a function’s docstring using the
# doctest module. Create a function add(a, b) that returns the sum of two numbers and includes
# multipletest cases in its docstring. Implement a way to automatically run the tests when the script is executed.
def add(a, b):
    """
    Returns the sum of two numbers.
    >>> add(2, 3)
    5
    >>> add(-1, 1)
    0
    >>> add(0, 0)
    0
    """
    return a + b

if __name__ == "__main__":
    import doctest
    doctest.testmod(verbose=True) #doctest.testmod() <-- for (.py) files
```

```
Trying:
    add(2, 3)
```

```
Expecting:
    5
ok
Trying:
    add(-1, 1)
Expecting:
    0
ok
Trying:
    add(0, 0)
Expecting:
    0
ok
1 item had no tests:
    __main__
1 item passed all tests:
   3 tests in __main__.add
3 tests in 2 items.
3 passed.
Test passed.
```

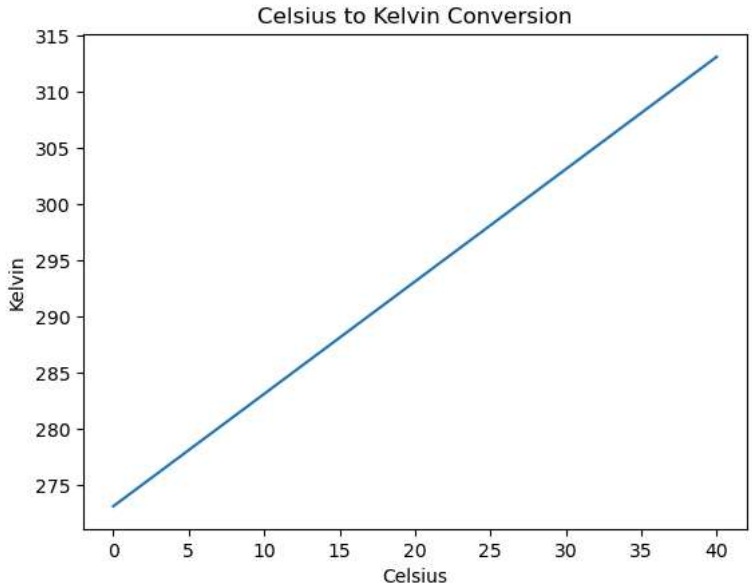
```
'''
Q18)Write a Python program using a lambda function to convert temperatures from Celsius to Kelvin,
store the data in a tabular format using pandas, and visualize the data using a plot.
'''
import pandas as pd
import matplotlib.pyplot as plt

c_to_k = lambda c: c + 273.15

celsius = [0, 10, 20, 30, 40]
kelvin = list(map(c_to_k, celsius))

df = pd.DataFrame({
    "Celsius": celsius,
    "Kelvin": kelvin
})

print(df)

plt.plot(df["Celsius"], df["Kelvin"])
plt.xlabel("Celsius")
plt.ylabel("Kelvin")
plt.title("Celsius to Kelvin Conversion")
plt.show()
```

```
   Celsius  Kelvin
0        0  273.15
1       10  283.15
2       20  293.15
3       30  303.15
4       40  313.15
```